

Functional Programming

Radu Răzvan Slăvescu

Technical University of Cluj-Napoca
Department of Computer Science

Outline

Lazy evaluation

Data streams in circular structures

Memoization via circular structures

Challenge: consecutive colors in a deck of cards

Challenge

Consider a deck of 52 cards, half of which are red (both hearts and diamonds are so) and the others black (spades and clubs). You shuffle the deck, cut the cards and start looking at them. Find out the probability (3 digits) to see at least one sequence of at least 5 cards of the same color (either red or black).

Your solution should scale for the case of k decks mixed up.

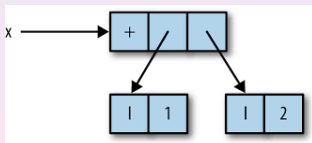
Lazy evaluation

Example (Lazy data)

```
Prelude> let x = 1+2 :: Int
```

```
Prelude> :sprint x -- print only, no evaluation
```

```
x = _      -- x is unevaluated, it is just a thunk
```



```
Prelude> x      -- this will force evaluation
3
```

```
Prelude> :sprint x
```

```
x = 3      -- thunk has been replaced by its value
```

Lazy evaluation

Example (Lazy data)

```
Prelude> let x = 1+2 :: Int
```

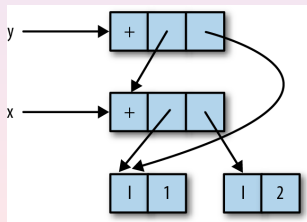
```
Prelude> let y = x+1 :: Int
```

```
Prelude> :sprint x
```

```
x = _
```

```
Prelude> :sprint y
```

```
y = _
```



Lazy evaluation

Example (Lazy data)

--seq evaluates arg #1 then returns arg #2

```
Prelude> seq y ()
```

```
()
```

```
Prelude> :sprint y
```

```
y = 4
```

```
Prelude> :sprint x
```

```
x = 3          -- thunk evaluated because of y
```

Lazy evaluation

Example (Lazy recursive data structures and WHNF)

--map builds a lazy data structure

```
map f [] = []
```

```
map f (x:xs) = x' : xs' where
```

```
    x' = f x      -- thunk
```

```
    xs' = map f xs -- also a thunk
```

```
Prelude> let xs = map (*2) [1..5] :: [Int]
```

```
Prelude> seq xs ()
```

```
()
```

```
Prelude> :sprint xs
```

```
xs = _ : _
```

--evaluate up to the first constructor

--xs is now in **weak head normal form (WHNF)**

Lazy evaluation

Weak Head Normal Form (WHNF)

An expression is in Weak Head Normal Form if it is either:

- ▶ a **constructor** (eventually applied to arguments), e.g.,
`True`, `Just 42` or `((:) 1)`
- ▶ or a **lambda abstraction** `\ x -> expression`
- ▶ a **built-in function** applied to too few arguments, e.g.,
`(+) 2` or `sqrt`

Adapted from: https://wiki.haskell.org/Weak_head_normal_form

Remark

If the outermost part of expression E is a constructor or a lambda abstraction, then E is in WHNF.

Lazy evaluation

Example (Lazy data structures)

```
Prelude> :sprint xs
```

```
xs = _ : _
```

```
Prelude> length xs
```

```
5
```

```
Prelude> :sprint xs
```

```
xs = [_,_,_,_,_] --structure generated
```

```
Prelude> sum xs
```

```
30
```

```
Prelude> :sprint xs
```

```
xs = [2,4,6,8,10]
```

(Examples from S. Marlow. *Parallel and Concurrent Programming in Haskell*. O'Reilly Media, 2013, Chapter 2)

Lazy evaluation

Definition (Function seq)

`seq x y` evaluates `x` to WHNF and returns `y`

Example (`foldl'`)

`foldl' o z [] = z`

`foldl' o z (x:xs) =`

`let`

`z' = z `o` x`

`in`

`seq z' $ foldl' o z' xs`

Lazy evaluation: one piece at a time



Customer: "I could use a couple of Cuba Libres"

Bar tender: "Here you are. The first one. I'm ready to prepare the next. Just let me know when you need it"

Lazy evaluation: infinite lists

Example (A (potentially) infinite list of 2s)

```
Prelude> twos = [2,2..]
```

```
Prelude> take 5 twos  
[2,2,2,2,2]
```

Remark

One can extract as many elements from this list as needed.

Lazy evaluation: infinite lists

Example (Turner's Eratosthenes-like sieve revisited)

```
sv (p:ns) = p : sv [ n | n <- ns, n `mod` p > 0 ]  
primes = sv [2..]
```

Trace sketch

```
sv [2..]  
= 2 : sv on all odd numbers (first: 3)  
= 2 : 3 : sv on all odd numbers not divisible by 3 (first: 5)  
= 2 : 3 : 5 : sv on all odd numbers not divisible by 3 or 5  
...
```

Evaluation termination? `primes`: no; a prefix of it: yes

```
Prelude> take 5 primes  
[2,3,5,7,11]
```

Lazy evaluation: only what we need

Example (Minimum of a list using insert sort)

```
sorti []      = []
sorti (x:xs) = ins x (sorti xs) where
  ins x []      = [x]
  ins x (y:ys) | x < y      = x:y:ys
                | otherwise = y:ins x ys

mymin xs = head (sorti xs)
```

Lazy evaluation: only what we need

Example (Tracing `mymin`)

```
sorti [] = []
sorti (x:xs) = ins x (sorti xs) where
  ins x [] = [x]
  ins x (y:ys) | x < y = x:y:ys
                | otherwise = y:ins x ys
mymin xs = head (sorti xs)
```

```
mymin [4,2,1,3] =
head (sorti [4,2,1,3]) =
head (ins 4 (sorti [2,1,3])) =
head (ins 4 (ins 2 (sorti [1,3]))) =
head (ins 4 (ins 2 (ins 1 (sorti [3]))) =
head (ins 4 (ins 2 (ins 1 (ins 3 (sorti [])))) =
head (ins 4 (ins 2 (ins 1 (ins 3 [])))) =
head (ins 4 (ins 2 (ins 1 (3:[])))) =
head (ins 4 (ins 2 (1:(3:[])))) =
head (ins 4 (1: (ins 2 (3:[])))) =
head (1: (ins 4 (ins 2 (3:[])))) =
```

Data streams in circular structures

Recursive call or circular data structure?

```
twos=2:twos
```

Remarks

Program = Network of processes

1. they produce and consume data streams
2. communication and timing taken care of by lazy evaluation
3. implementation close to problem specification

Understanding circular structures

Definition (Fibonacci, take 1)

1. $n_0 = 0$
2. $n_1 = 1$
3. $n_k = n_{k-1} + n_{k-2}$ for $k \geq 2$

Definition (Fibonacci sequences)

We'll call f_i the sequence of Fibonacci numbers starting from position i . We'll have:

- ▶ $f_0 = n_0, n_1, n_2, \dots = 0 : f_1$
- ▶ $f_1 = n_1, n_2, n_3, \dots = 1 : f_2$
- ▶ $f_2 = n_2, n_3, n_4, \dots = \text{sum of } f_0 \text{ and } f_1 \text{ element by element}$

Understanding circular structures

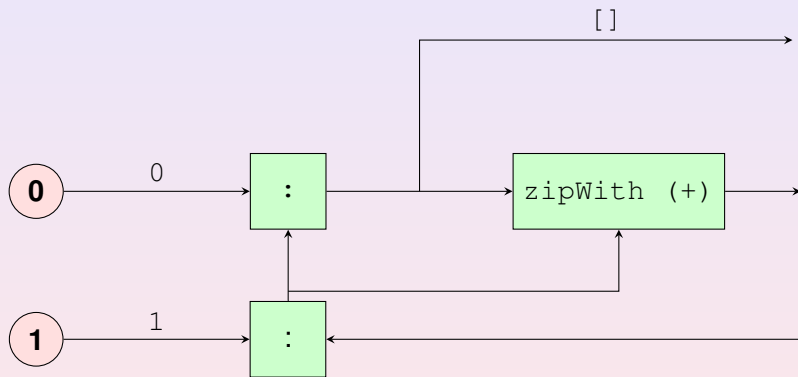
Definition (Fibonacci sequences)

- ▶ $f_0 = n_0, n_1, n_2, \dots = 0 : f_1$
- ▶ $f_1 = n_1, n_2, n_3, \dots = 1 : f_2$
- ▶ $f_2 = n_2, n_3, n_4, \dots = \text{sum of } f_0 \text{ and } f_1 \text{ element by element}$

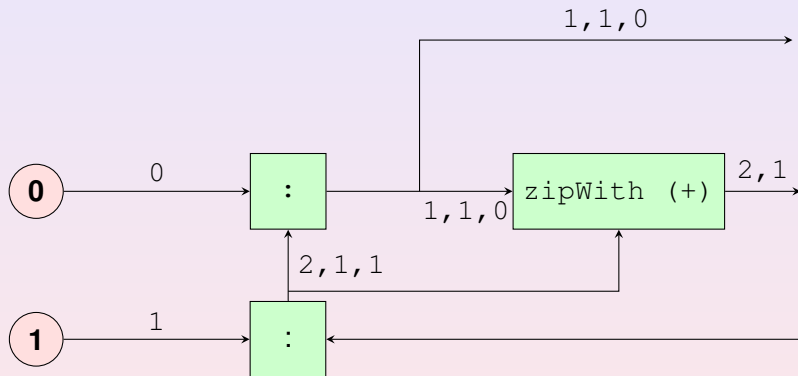
Example (Generating Fibonacci numbers sequences)

```
fibonacciGen n = take n f0 where  
  f0 = 0:f1  
  f1 = 1:f2  
  f2 = zipWith (+) f0 f1
```

A circular structure for Fibonacci numbers



A circular structure for Fibonacci numbers



A circular structure for the Hamming numbers

Definition (Hamming numbers sequence)

It's a strictly increasing sequence of natural numbers such that:

1. 1 is member of the sequence
2. if n is in the sequence, then so are $2*n$, $3*n$ and $5*n$
3. no other number is in the sequence

E.g. 1,2,3,4,5,6,8,9,10,12,15,16,18,20,24,25,27,30,32,36,...

Example (Generating Hamming numbers)

```
hammingGen n = take n ham where  
  ham = 1:merge3 [ 2*i | i <- ham ]  
                [ 3*i | i <- ham ]  
                [ 5*i | i <- ham ]
```

A circular structure for Hamming numbers

Example (Generating Hamming numbers: merging 3 lists)

```
merge3 xs ys zs = merge (merge xs ys) zs where
  merge (u:us) (v:vs)
    | u < v      = u:merge us (v:vs)
    | u > v      = v:merge (u:us) vs
    | otherwise = u:merge us vs
```

Example (The first 16 Hamming numbers)

```
Main> hammingGen 16
[1,2,3,4,5,6,8,9,10,12,15,16,18,20,24,25]
```

A circular structure for the Hamming numbers

Example (Generating Hamming numbers)

```
hammingGen n = take n ham where
  ham = 1:merge3 [ 2*i | i <- ham ]
                [ 3*i | i <- ham ]
                [ 5*i | i <- ham ]
```

Remark

3 producers (3 list comprehensions) feed 1 consumer
No programming effort for communication

Well-founded definitions

Example (Some circular definitions are well founded...)

```
mgood = 1:head (tail (tail mgood)):mgood
```

```
Main> take 5 mgood  
[1,1,1,1,1]
```

Example (...but some others are not)

```
mbad = 1:head (tail (tail (tail mbad))):mbad
```

```
Main> take 5 mbad  
[1,      {Interrupted!}]
```

Your turn

Example (Dancing Cha-cha-cha)

```
li = 1 : 4*el : 2*el : 5*el : 3*el : li where  
    el = head (tail (tail (tail (tail (tail li))))))
```

```
foo = take 9 (map espagnol (tail li)) where  
    espagnol e | e == 1      = "uno"  
               | e == 2      = "dos"  
               | e == 3      = "tres"  
               | otherwise = "chacha"
```

```
*Main> foo
```

```
["chacha", "dos", "chacha", "tres", "uno",  
 "chacha", "dos", "chacha", "tres"]
```

Memoization

Definition (Memoization)

Cache the values computed and reuse them.

Example (Fibonacci, take 2)

```
Prelude> fi = 0:1:zipWith (+) fi (tail fi) :: [Int]
```

```
Prelude> :sprint fi
```

```
fi = _
```

```
Prelude> fi !! 4
```

```
3
```

```
Prelude> :sprint fi
```

```
fi = 0 : 1 : 1 : 2 : 3 : _ memoize computed results
```

Memoization

Example (Fibonacci, take 2)

```
Prelude> fi = 0:1:zipWith (+) fi (tail fi) :: [Int]
Prelude> fi !! 4
3
Prelude> :sprint fi
Prelude> fi = 0 : 1 : 1 : 2 : 3 : _
```

Remark

Lazy evaluation together with circular structures allow elegant memoization, sometimes with zero effort from the programmer.

Polygon triangulation once again $T_n = \sum_{i=2}^{n-1} T_i * T_{n+1-i}$

Example (Haskell v2: cache in a list accumulator)

```
t2 n = t2a 1 [1] where
  t2a k cs
    | k == n-1  = head cs
    | otherwise = t2a (k+1) (cn:cs) where
                        cn = sum (zipWith (*)
                                cs (reverse cs))
```

Example (Haskell v3: memoize via circular lists)

```
ntrsqs = 1:1:map (\ n -> sum (comb n)) [4..] where
  comb n = let
    uptos = take (n-2) ntrsqs
  in
    zipWith (*) uptos (reverse uptos)
```

Pascal Triangle

Example (Building the triangle)

Row 0: just a 1

Row k : add the two neighbors just above (if absent, assume 0)

```
    1
   1 1
  1 2 1
 1 3 3 1
1 4 6 4 1
```

Pascal Triangle

Example (Build the triangle: `map` on a circular structure)

```
pascalSeq =  
  let  
    infix 6 &:  
    x &: xs = xs ++ [x]  
  in  
    [1] : (map (\r -> zipWith (+) (0:r) (0&:r)))  
          pascalSeq
```

Example (A one-liner for the triangle)

```
ps=[1]:map (\r->zipWith (+) (0:r) (r++[0])) ps
```

Summary

1. Lazy evaluation allows data potentially infinite
2. Lazy evaluation allows building circular structures. Data streams produced and used in processing networks
3. Code: elegant, concise, close to problem spec
4. A tradeoff with the overhead of the interpreter, garbage collector and debugger is needed

For those **eager** to try something new ...



G. Hutton

A Haskell implementation of Sudoku using lazy evaluation

<http://www.cs.nott.ac.uk/~pszgmh/sudoku.lhs>